Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Comentado [PM1]: https://github.com/Pascoalpm/lab-numpy

Институт Перспективной инженерии
Департамент цифровых и робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2
дисциплины «Искусственный Интеллект и Машинное Обучение»

Выполнил: Мендеш Пашкоал Педру
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и Вычислительная техника», направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Роман А. доцент факультета цифровых, робототехнических систем и электроники института перспективной инженерии.

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Основы работы с библиотекой NumPy

Цель работы: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Методика выполнения работы

- Изучение теоретического материала
- Создание репозитория GitHub: https://github.com/Pascoalpm/lab-numpy
- Клонирование репозитория
- Настройка .gitignore
- Организация ветвления git-flow
- Решение практических заданий
- Изучение примеров практической работы
- Фиксация изменений в репозитории
- Отправка изменений на сервер GitHub
- Загрузка отчета в формате PDF

Примеры практической работы

1: Создание и доступ к элементам матрицы

```
import numpy as np

# Создание матрицы
m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
print("Исходная матрица:")
print(m)

# Доступ к элементам
print("\nЭлемент m[1,0]:", m[1,0])
print("Вторая строка:", m[1, :])
print("Третий столбец:", m[:, 2])
print("Часть строки (со 2-го элемента):", m[1, 2:])
```

```
Исходная матрица:
[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]

Элемент m[1,0]: 5
Вторая строка: [[5 6 7 8]]
Третий столбец: [[3]
 [7]
 [5]]
Часть строки (со 2-го элемента): [[7 8]]
```

Рисунок 1. Примр 1

2: Статистические функции NumPy

```python
import numpy as np

# Создание матрицы
m = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 1, 5, 7]])

print("Матрица:")
print(m)
print(f"Размерность (shape): {m.shape}")
print(f"\nМаксимальный элемент (все): {m.max()}")
print(f"Максимум по строкам (axis=1): {m.max(axis=1)}")
print(f"Максимум по столбцам (axis=0): {m.max(axis=0)}")
print(f"\nСреднее значение: {m.mean():.2f}")
print(f"Среднее по строкам: {m.mean(axis=1)}")
print(f"Среднее по столбцам: {m.mean(axis=0)}")
print(f"\nСумма всех элементов: {m.sum()}")
print(f"Стандартное отклонение: {m.std():.2f}")
```

```
Матрица:
[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
Размерность (shape): (3, 4)

Максимальный элемент (все): 9
Максимум по строкам (axis=1): [4 8 9]
Максимум по столбцам (axis=0): [9 6 7 8]

Среднее значение: 4.83
Среднее по строкам: [2.5 6.5 5.5]
Среднее по столбцам: [5.         3.         5.         6.33333333]

Сумма всех элементов: 58
Стандартное отклонение: 2.58
```

Рисунок 2. Пример 2

3: Булевы маски и фильтрация данных

```python
import numpy as np

# Создание массива
nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Исходный массив:", nums)

# Создание булевой маски
less_then_5 = nums < 5
print("Маска (nums < 5):", less_then_5)

# Фильтрация с помощью маски
filtered = nums[less_then_5]
print("Элементы < 5:", filtered)

# Замена элементов по условию
nums[nums < 5] = 10
print("Массив после замены (<5 → 10):", nums)

# Работа с двумерным массивом
m = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 1, 5, 7]])

# Сложное условие (элементы от 3 до 7)
mask = np.logical_and(m >= 3, m <= 7)
print("\nИсходная матрица:")
print(m)
print("Маска (3 ≤ элемент ≤ 7):")
print(mask)
print("Элементы, удовлетворяющие условию:")
print(m[mask])
```

```
Исходный массив: [ 1  2  3  4  5  6  7  8  9 10]
Маска (nums < 5): [ True  True  True  True False False False False False False]
Элементы < 5: [1 2 3 4]
Массив после замены (<5 → 10): [10 10 10 10  5  6  7  8  9 10]

Исходная матрица:
[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
Маска (3 ≤ элемент ≤ 7):
[[False False  True  True]
 [ True  True  True False]
 [False False  True  True]]
Элементы, удовлетворяющие условию:
[3 4 5 6 7 5 7]
```

Рисунок 3. Пример 3

4: Специальные функции создания массивов

```python
# np.arange - создание последовательности
print("1. np.arange(10):", np.arange(10))
print("2. np.arange(5, 15):", np.arange(5, 15))
print("3. np.arange(1, 10, 1.5):", np.arange(1, 10, 1.5))

# np.zeros и np.eye
print("\n4. np.zeros((3,4)) - нулевая матрица:")
print(np.zeros((3, 4)))
print("\n5. np.eye(4) - единичная матрица:")
print(np.eye(4))

# np.diag - диагональная матрица
print("\n6. np.diag([5, 10, 15, 20]) - диагональная матрица:")
print(np.diag([5, 10, 15, 20]))

# np.ravel - преобразование в одномерный массив
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("\n7. Исходная матрица 3×3:")
print(A)
print("   np.ravel(A) (порядок 'C' - по строкам):", np.ravel(A))
print("   np.ravel(A, order='F') (по столбцам):", np.ravel(A, order='F'))

# np.random.permutation - перемешивание
print("\n8. np.random.permutation(10):", np.random.permutation(10))
arr = np.array([10, 20, 30, 40, 50])
print("   Исходный массив:", arr)
print("   Перемешанный:", np.random.permutation(arr))
```

```
1. np.arange(10): [0 1 2 3 4 5 6 7 8 9]
2. np.arange(5, 15): [ 5  6  7  8  9 10 11 12 13 14]
3. np.arange(1, 10, 1.5): [1.  2.5 4.  5.5 7.  8.5]

4. np.zeros((3,4)) - нулевая матрица:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

5. np.eye(4) - единичная матрица:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

6. np.diag([5, 10, 15, 20]) - диагональная матрица:
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]

7. Исходная матрица 3×3:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
   np.ravel(A) (порядок 'C' - по строкам): [1 2 3 4 5 6 7 8 9]
   np.ravel(A, order='F') (по столбцам): [1 4 7 2 5 8 3 6 9]

8. np.random.permutation(10): [5 3 7 0 9 2 1 6 4 8]
   Исходный массив: [10 20 30 40 50]
   Перемешанный: [30 10 20 50 40]
```

Рисунок 4. Примр 4

5: Операции с матрицами

```python
import numpy as np

# Создание матриц
A = np.array([[1, 2, 3],
              [4, 5, 6]])

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])

C = np.array([[1, 2],
              [3, 4]])

print("=== УМНОЖЕНИЕ МАТРИЦЫ НА ЧИСЛО ===")
print("Матрица A:")
print(A)
print("3 × A:")
print(3 * A)

print("\n=== СЛОЖЕНИЕ МАТРИЦ ===")
D = np.array([[5, 6], [7, 8]])
print("C + D:")
print(C + D)

print("\n=== МАТРИЧНОЕ УМНОЖЕНИЕ ===")
print("A (2×3) × B (3×2) =")
result = A @ B
print(result)
print(f"Размерность результата: {result.shape}")

print("\n=== ДЕТАЛЬНЫЙ РАСЧЕТ ===")
print("result[0,0] = 1×7 + 2×9 + 3×11 =", 1*7 + 2*9 + 3*11)
print("result[0,1] = 1×8 + 2×10 + 3×12 =", 1*8 + 2*10 + 3*12)
print("result[1,0] = 4×7 + 5×9 + 6×11 =", 4*7 + 5*9 + 6*11)
print("result[1,1] = 4×8 + 5×10 + 6×12 =", 4*8 + 5*10 + 6*12)

print("\n=== ПРОВЕРКА НЕКОММУТАТИВНОСТИ ===")
print("C × D:")
print(C @ D)
print("D × C:")
print(D @ C)
print("C × D = D × C?", np.array_equal(C @ D, D @ C))
```

Рисунок 5. Примр 5 (код)

```
=== УМНОЖЕНИЕ МАТРИЦЫ НА ЧИСЛО ===
Матрица A:
[[1 2 3]
 [4 5 6]]
3 × A:
[[ 3  6  9]
 [12 15 18]]

=== СЛОЖЕНИЕ МАТРИЦ ===
C + D:
[[ 6  8]
 [10 12]]

=== МАТРИЧНОЕ УМНОЖЕНИЕ ===
A (2×3) × B (3×2) =
[[ 58  64]
 [139 154]]
Размерность результата: (2, 2)

=== ДЕТАЛЬНЫЙ РАСЧЕТ ===
result[0,0] = 1×7 + 2×9 + 3×11 = 58
result[0,1] = 1×8 + 2×10 + 3×12 = 64
result[1,0] = 4×7 + 5×9 + 6×11 = 139
result[1,1] = 4×8 + 5×10 + 6×12 = 154

=== ПРОВЕРКА НЕКОММУТАТИВНОСТИ ===
C × D:
[[19 22]
 [43 50]]
D × C:
[[23 34]
 [31 46]]
C × D = D × C? False
```

Рисунок 6. Пример 5

6: Транспонирование матриц

```python
import numpy as np

# Создание матрицы
A = np.array([[1, 2, 3],
              [4, 5, 6]])

print("Исходная матрица A (2×3):")
print(A)

# Разные способы транспонирования
print("\n=== СПОСОБЫ ТРАНСПОНИРОВАНИЯ ===")
print("A.T:")
print(A.T)
print("np.transpose(A):")
print(np.transpose(A))
print("A.transpose():")
print(A.transpose())

# Свойства транспонирования
print("\n=== СВОЙСТВА ТРАНСПОНИРОВАНИЯ ===")
print("Свойство 1: (A^T)^T = A")
print((A.T).T)
print("Равны?", np.array_equal(A, (A.T).T))

B = np.array([[7, 8, 9],
              [10, 11, 12]])

print("\nСвойство 2: (A + B)^T = A^T + B^T")
print("(A + B)^T:")
print((A + B).T)
print("A^T + B^T:")
print(A.T + B.T)

# След матрицы
print("\n=== СЛЕД МАТРИЦЫ (trace) ===")
C = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print("Матрица C:")
print(C)
print(f"След C (np.trace): {np.trace(C)}")
print(f"След C (ручной расчет): {C[0,0] + C[1,1] + C[2,2]}")
print(f"След C^T: {np.trace(C.T)}")
print(f"След C = след C^T? {np.trace(C) == np.trace(C.T)}")
```

Рисунок 7. Пример 6 (код)

```
Исходная матрица A (2×3):
[[1 2 3]
 [4 5 6]]

=== СПОСОБЫ ТРАНСПОНИРОВАНИЯ ===
A.T:
[[1 4]
 [2 5]
 [3 6]]
np.transpose(A):
[[1 4]
 [2 5]
 [3 6]]
A.transpose():
[[1 4]
 [2 5]
 [3 6]]

=== СВОЙСТВА ТРАНСПОНИРОВАНИЯ ===
Свойство 1: (A^T)^T = A
[[1 2 3]
 [4 5 6]]
Равны? True

Свойство 2: (A + B)^T = A^T + B^T
(A + B)^T:
[[ 8 14]
 [10 16]
 [12 18]]
A^T + B^T:
[[ 8 14]
 [10 16]
 [12 18]]

=== СЛЕД МАТРИЦЫ (trace) ===
Матрица C:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
След C (np.trace): 15
След C (ручной расчет): 15
След C^T: 15
След C = след C^T? True
```

Рисунок 8. Пример 6

7: Определитель и обратная матрица

```python
import numpy as np

# Создание матрицы 2×2
A = np.array([[1, 2],
              [3, 4]])

print("=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ 2×2 ===")
print("Матрица A:")
print(A)
det_A = np.linalg.det(A)
print(f"det(A) = 1×4 - 2×3 = {det_A}")

# Создание матрицы 3×3
B = np.array([[4, -1, 2],
              [10, 4, -1],
              [8, 3, 1]])

print("\n=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ 3×3 ===")
print("Матрица B:")
print(B)
det_B = np.linalg.det(B)
print(f"det(B) = {det_B:.0f}")

print("\n=== ОБРАТНАЯ МАТРИЦА ===")
print("Матрица A:")
print(A)
print(f"det(A) = {det_A} ≠ 0, обратная матрица существует")

A_inv = np.linalg.inv(A)
print("A^(-1):")
print(A_inv)

print("\n=== ПРОВЕРКА: A × A^(-1) ===")
print(A @ A_inv)
print("Должна быть единичная матрица:")
print(np.eye(2))

print("\n=== СВОЙСТВА ОБРАТНОЙ МАТРИЦЫ ===")
print("Свойство 1: (A^(-1))^(-1) = A")
print(np.linalg.inv(A_inv))

print("\nСвойство 2: det(A^(-1)) = 1 / det(A)")
print(f"det(A^(-1)) = {np.linalg.det(A_inv):.2f}")
print(f"1 / det(A) = {1/det_A:.2f}")
```

Рисунок 9. Пример 7 (код)

```
=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ 2×2 ===
Матрица A:
[[1 2]
 [3 4]]
det(A) = 1×4 - 2×3 = -2.0000000000000004

=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ 3×3 ===
Матрица B:
[[ 4 -1  2]
 [10  4 -1]
 [ 8  3  1]]
det(B) = 42

=== ОБРАТНАЯ МАТРИЦА ===
Матрица A:
[[1 2]
 [3 4]]
det(A) = -2.0000000000000004 ≠ 0, обратная матрица существует
A^(-1):
[[-2.   1. ]
 [ 1.5 -0.5]]

=== ПРОВЕРКА: A × A^(-1) ===
[[1.0000000e+00 0.0000000e+00]
 [8.8817842e-16 1.0000000e+00]]
Должна быть единичная матрица:
[[1. 0.]
 [0. 1.]]

=== СВОЙСТВА ОБРАТНОЙ МАТРИЦЫ ===
Свойство 1: (A^(-1))^(-1) = A
[[1. 2.]
 [3. 4.]]

Свойство 2: det(A^(-1)) = 1 / det(A)
det(A^(-1)) = -0.50
1 / det(A) = -0.50
```

Рисунок 10. Пример 7

Задания практической работы

1: Создание и изменение массивов

Создайте массив NumPy размером 3×3, содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

```python
# Упражнение 1 - Создание и изменение массивов
import numpy as np

# 1. Создаем массив 3x3 с числами от 1 до 9
originalniy_array = np.arange(1, 10).reshape(3, 3)
print("=== ИСХОДНЫЙ МАССИВ ===")
print(originalniy_array)
print(f"Размерность: {originalniy_array.shape}")
print()

# 2. Умножаем все элементы на 2
umnozhenniy_array = originalniy_array * 2
print("=== ПОСЛЕ УМНОЖЕНИЯ НА 2 ===")
print(umnozhenniy_array)
print()

# 3. Заменяем элементы больше 10 на 0
konechniy_array = umnozhenniy_array.copy()  # Создаем копию
konechniy_array[konechniy_array > 10] = 0
print("=== ИТОГОВЫЙ МАССИВ (элементы > 10 заменены на 0) ===")
print(konechniy_array)
print()

# 4. Дополнительные проверки
print("=== ПРОВЕРКИ ===")
print(f"Исходные элементы: {originalniy_array.flatten()}")
print(f"После умножения: {umnozhenniy_array.flatten()}")
print(f"Итоговые элементы: {konechniy_array.flatten()}")
print(f"Количество нулей в итоговом массиве: {np.sum(konechniy_array == 0)}")
```

Рисунок 11. Код, выполненный в Notebook

```
=== ИСХОДНЫЙ МАССИВ ===
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Размерность: (3, 3)

=== ПОСЛЕ УМНОЖЕНИЯ НА 2 ===
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]

=== ИТОГОВЫЙ МАССИВ (элементы > 10 заменены на 0) ===
[[ 2  4  6]
 [ 8 10  0]
 [ 0  0  0]]

=== ПРОВЕРКИ ===
Исходные элементы: [1 2 3 4 5 6 7 8 9]
После умножения: [ 2  4  6  8 10 12 14 16 18]
Итоговые элементы: [ 2  4  6  8 10  0  0  0  0]
Количество нулей в итоговом массиве: 4
```

Рисунок 12. Результат выполнения кода в Notebook

2: Работа с булевыми масками

Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

```python
# Упражнение 2 - Работа с булевыми масками
import numpy as np

# 1. Генерируем массив с 20 случайными целыми числами от 1 до 100
np.random.seed(42)  # Для воспроизводимости результатов
originalniy_array = np.random.randint(1, 101, size=20)

print("=== ИСХОДНЫЙ МАССИВ ===")
print(originalniy_array)
print()

# 2. Находим элементы, делящиеся на 5
delitsya_na_5 = originalniy_array[originalniy_array % 5 == 0]
print("=== ЭЛЕМЕНТЫ, ДЕЛЯЩИЕСЯ НА 5 ===")
print(delitsya_na_5)
print(f"Количество: {len(delitsya_na_5)} элементов")
print()

# 3. Заменяем эти элементы на -1
izmenenniy_array = originalniy_array.copy()
izmenenniy_array[izmenenniy_array % 5 == 0] = -1

print("=== МАССИВ ПОСЛЕ ЗАМЕНЫ (делящиеся на 5 → -1) ===")
print(izmenenniy_array)
print()

# 4. Сравнение исходного и измененного массивов
print("=== СРАВНЕНИЕ ИСХОДНЫЙ х ИЗМЕНЕННЫЙ ===")
print("Исходный:  ", originalniy_array)
print("Измененный:", izmenenniy_array)
print()

# 5. Дополнительная статистика
print("=== СТАТИСТИКА ===")
print(f"Среднее исходного массива: {np.mean(originalniy_array):.2f}")
print(f"Среднее измененного массива: {np.mean(izmenenniy_array):.2f}")
print(f"Минимальный элемент (исходный): {np.min(originalniy_array)}")
print(f"Максимальный элемент (исходный): {np.max(originalniy_array)}")
```

Рисунок 13. Код, выполненный в Notebook

```
=== ИСХОДНЫЙ МАССИВ ===
[ 52  93  15  72  61  21  83  87  75  75  88 100  24   3  22  53   2  88
  30  38]

=== ЭЛЕМЕНТЫ, ДЕЛЯЩИЕСЯ НА 5 ===
[ 15  75  75 100  30]
Количество: 5 элементов

=== МАССИВ ПОСЛЕ ЗАМЕНЫ (делящиеся на 5 → -1) ===
[52 93 -1 72 61 21 83 87 -1 -1 88 -1 24  3 22 53  2 88 -1 38]

=== СРАВНЕНИЕ ИСХОДНЫЙ x ИЗМЕНЕННЫЙ ===
Исходный:   [ 52  93  15  72  61  21  83  87  75  75  88 100  24   3  22  53   2  88
  30  38]
Измененный: [52 93 -1 72 61 21 83 87 -1 -1 88 -1 24  3 22 53  2 88 -1 38]

=== СТАТИСТИКА ===
Среднее исходного массива: 54.10
Среднее измененного массива: 39.10
Минимальный элемент (исходный): 2
Максимальный элемент (исходный): 100
```

Рисунок 14. Результат выполнения кода в Notebook

3: Объединение и разделение массивов

Создайте два массива NumPy размером 1×5, заполненных случайными числами от 0 до 50.

- Объедините эти массивы в один двумерный массив (по строкам).
- Разделите полученный массив на два массива, каждый из которых содержит 5 элементов.
- Выведите все промежуточные и итоговые результаты.

```python
# Упражнение 3 - Объединение и разделение массивов
import numpy as np

# 1. Создаем два массива 1x5 со случайными числами от 0 до 50
np.random.seed(42)  # Для воспроизводимости результатов
array1 = np.random.randint(0, 51, size=5)
array2 = np.random.randint(0, 51, size=5)

print("=== ИСХОДНЫЕ МАССИВЫ ===")
print(f"Массив 1: {array1}")
print(f"Массив 2: {array2}")
print(f"Размерность массива 1: {array1.shape}")
print(f"Размерность массива 2: {array2.shape}")
print()

# 2. Объединяем массивы в один двумерный (по строкам)
# Используем np.vstack() для вертикального объединения (по строкам)
obedinenny = np.vstack((array1, array2))

print("=== ОБЪЕДИНЕННЫЙ МАССИВ (по строкам) ===")
print(obedinenny)
print(f"Размерность объединенного массива: {obedinenny.shape}")
print(f"Количество строк: {obedinenny.shape[0]}")
print(f"Количество столбцов: {obedinenny.shape[1]}")
print()

# 3. Разделяем полученный массив на два массива по 5 элементов
# Используем np.vsplit() для разделения по вертикали
array1_razdelenny, array2_razdelenny = np.vsplit(obedinenny, 2)

print("=== РАЗДЕЛЕННЫЕ МАССИВЫ ===")
print("Первый разделенный массив:")
print(array1_razdelenny)
print(f"Размерность: {array1_razdelenny.shape}")

print("\nВторой разделенный массив:")
print(array2_razdelenny)
print(f"Размерность: {array2_razdelenny.shape}")
print()
```

Рисунок 15. Код, выполненный в Notebook

```
=== ИСХОДНЫЕ МАССИВЫ ===
Массив 1: [38 28 14 42  7]
Массив 2: [20 38 18 22 10]
Размерность массива 1: (5,)
Размерность массива 2: (5,)

=== ОБЪЕДИНЕННЫЙ МАССИВ (по строкам) ===
[[38 28 14 42  7]
 [20 38 18 22 10]]
Размерность объединенного массива: (2, 5)
Количество строк: 2
Количество столбцов: 5

=== РАЗДЕЛЕННЫЕ МАССИВЫ ===
Первый разделенный массив:
[[38 28 14 42  7]]
Размерность: (1, 5)

Второй разделенный массив:
[[20 38 18 22 10]]
Размерность: (1, 5)

=== СРАВНЕНИЕ С ОРИГИНАЛОМ ===
Оригинальный массив 1: [38 28 14 42  7]
Разделенный массив 1 (преобразованный): [38 28 14 42  7]
Оригинальный массив 2: [20 38 18 22 10]
Разделенный массив 2 (преобразованный): [20 38 18 22 10]

=== ПРОВЕРКА СООТВЕТСТВИЯ ===
Массив 1 совпадает с оригиналом: True
Массив 2 совпадает с оригиналом: True
```

Рисунок 16. Результат выполнения кода в Notebook

4: Генерация и работа с линейными последовательностями

Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

```python
# Упражнение 4 - Генерация и работа с линейными последовательностями
import numpy as np

# 1. Создаем массив из 50 чисел, равномерно распределенных от -10 до 10
# Используем np.linspace() для равномерного распределения
array = np.linspace(-10, 10, 50)

print("=== ИСХОДНЫЙ МАССИВ ===")
print(f"Первые 10 элементов: {array[:10]}")
print(f"Последние 10 элементов: {array[-10:]}")
print(f"Размерность массива: {array.shape}")
print(f"Количество элементов: {len(array)}")
print(f"Минимальное значение: {np.min(array):.2f}")
print(f"Максимальное значение: {np.max(array):.2f}")
print()

# 2. Вычисляем сумму всех элементов
summa_vseh = np.sum(array)
print("=== СУММА ВСЕХ ЭЛЕМЕНТОВ ===")
print(f"Сумма всех элементов: {summa_vseh:.2f}")
print()

# 3. Вычисляем сумму положительных элементов (> 0)
polozhitelnye = array[array > 0]
summa_polozhitelnyh = np.sum(polozhitelnye)
print("=== СУММА ПОЛОЖИТЕЛЬНЫХ ЭЛЕМЕНТОВ ===")
print(f"Положительные элементы (первые 10): {polozhitelnye[:10]}")
print(f"Количество положительных элементов: {len(polozhitelnye)}")
print(f"Сумма положительных элементов: {summa_polozhitelnyh:.2f}")
print()

# 4. Вычисляем сумму отрицательных элементов (< 0)
otricatelnye = array[array < 0]
summa_otricatelnyh = np.sum(otricatelnye)
print("=== СУММА ОТРИЦАТЕЛЬНЫХ ЭЛЕМЕНТОВ ===")
print(f"Отрицательные элементы (первые 10): {otricatelnye[:10]}")
print(f"Количество отрицательных элементов: {len(otricatelnye)}")
print(f"Сумма отрицательных элементов: {summa_otricatelnyh:.2f}")
print()
```

```python
# 5. Проверка: сумма всех = сумма положительных + сумма отрицательных
print("=== ПРОВЕРКА ===")
print(f"Сумма всех элементов: {summa_vseh:.2f}")
print(f"Сумма положительных + сумма отрицательных: {summa_polozhitelnyh + summa_otricatelnyh:.2f}")
print(f"Равенство выполняется: {np.isclose(summa_vseh, summa_polozhitelnyh + summa_otricatelnyh)}")
print()

# 6. Дополнительная статистика
print("=== ДОПОЛНИТЕЛЬНАЯ СТАТИСТИКА ===")
print(f"Среднее значение: {np.mean(array):.2f}")
print(f"Медиана: {np.median(array):.2f}")
print(f"Стандартное отклонение: {np.std(array):.2f}")
print(f"Дисперсия: {np.var(array):.2f}")
```

Рисунок 17. Код, выполненный в Notebook

```
=== ИСХОДНЫЙ МАССИВ ===
Первые 10 элементов: [-10.         -9.59183673  -9.18367347  -8.7755102   -8.36734694
  -7.95918367  -7.55102041  -7.14285714  -6.73469388  -6.32653061]
Последние 10 элементов: [ 6.32653061  6.73469388  7.14285714  7.55102041  7.95918367  8.36734694
  8.7755102   9.18367347  9.59183673 10.        ]
Размерность массива: (50,)
Количество элементов: 50
Минимальное значение: -10.00
Максимальное значение: 10.00

=== СУММА ВСЕХ ЭЛЕМЕНТОВ ===
Сумма всех элементов: 0.00

=== СУММА ПОЛОЖИТЕЛЬНЫХ ЭЛЕМЕНТОВ ===
Положительные элементы (первые 10): [0.20408163 0.6122449  1.02040816 1.42857143 1.83673469 2.24489796
 2.65306122 3.06122449 3.46938776 3.87755102]
Количество положительных элементов: 25
Сумма положительных элементов: 127.55

=== СУММА ОТРИЦАТЕЛЬНЫХ ЭЛЕМЕНТОВ ===
Отрицательные элементы (первые 10): [-10.         -9.59183673  -9.18367347  -8.7755102   -8.36734694
  -7.95918367  -7.55102041  -7.14285714  -6.73469388  -6.32653061]
Количество отрицательных элементов: 25
Сумма отрицательных элементов: -127.55

=== ПРОВЕРКА ===
Сумма всех элементов: 0.00
Сумма положительных + сумма отрицательных: 0.00
Равенство выполняется: True

=== ДОПОЛНИТЕЛЬНАЯ СТАТИСТИКА ===
Среднее значение: 0.00
Медиана: 0.00
Стандартное отклонение: 5.89
Дисперсия: 34.69
```

Рисунок 18. Результат выполнения кода в Notebook

5: Работа с диагональными и единичными матрицами

Создайте:

- Единичную матрицу размером 4×4.
- Диагональную матрицу размером 4×4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы).

Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

```python
# Упражнение 5 - Работа с диагональными и единичными матрицами
import numpy as np

# 1. Создаем единичную матрицу 4x4
edinichnaya = np.eye(4)
print("=== ЕДИНИЧНАЯ МАТРИЦА 4x4 ===")
print(edinichnaya)
print(f"Размерность: {edinichnaya.shape}")
print()

# 2. Создаем диагональную матрицу 4x4 с элементами [5, 10, 15, 20]
# Используем np.diag() для создания диагональной матрицы
diagonalnaya = np.diag([5, 10, 15, 20])
print("=== ДИАГОНАЛЬНАЯ МАТРИЦА 4x4 ===")
print(diagonalnaya)
print(f"Размерность: {diagonalnaya.shape}")
print()

# 3. Находим сумму всех элементов единичной матрицы
summa_edinichnaya = np.sum(edinichnaya)
print("=== СУММА ЭЛЕМЕНТОВ ЕДИНИЧНОЙ МАТРИЦЫ ===")
print(f"Сумма всех элементов: {summa_edinichnaya}")
print(f"Сумма элементов на главной диагонали (след): {np.trace(edinichnaya)}")
print()

# 4. Находим сумму всех элементов диагональной матрицы
summa_diagonalnaya = np.sum(diagonalnaya)
print("=== СУММА ЭЛЕМЕНТОВ ДИАГОНАЛЬНОЙ МАТРИЦЫ ===")
print(f"Сумма всех элементов: {summa_diagonalnaya}")
print(f"Сумма элементов на главной диагонали (след): {np.trace(diagonalnaya)}")
print()

# 5. Сравниваем результаты
print("=== СРАВНЕНИЕ РЕЗУЛЬТАТОВ ===")
print(f"Сумма элементов единичной матрицы: {summa_edinichnaya}")
print(f"Сумма элементов диагональной матрицы: {summa_diagonalnaya}")
print(f"Разница: {abs(summa_diagonalnaya - summa_edinichnaya)}")
print()

# 6. Детальный анализ
print("=== ДЕТАЛЬНЫЙ АНАЛИЗ ===")
print("Единичная матрица - элементы на диагонали:")
print(np.diag(edinichnaya))
print("Единичная матрица - элементы вне диагонали:")
print(f"Количество нулей вне диагонали: {np.sum(edinichnaya == 0)}")

print("\nДиагональная матрица - элементы на диагонали:")
print(np.diag(diagonalnaya))
print("Диагональная матрица - элементы вне диагонали:")
print(f"Количество нулей вне диагонали: {np.sum(diagonalnaya == 0)}")
print()

# 7. Альтернативные способы создания
print("=== АЛЬТЕРНАТИВНЫЕ СПОСОБЫ СОЗДАНИЯ ===")

# Единичная матрица через identity
edinichnaya_alt = np.identity(4)
```


```python
# Единичная матрица через identity
edinichnaya_alt = np.identity(4)
print("Единичная через np.identity(4):")
print(edinichnaya_alt)

# Диагональная матрица через zeros и заполнение диагонали
diagonalnaya_alt = np.zeros((4, 4))
np.fill_diagonal(diagonalnaya_alt, [5, 10, 15, 20])
print("\nДиагональная через np.zeros и fill_diagonal:")
print(diagonalnaya_alt)
```


Рисунок 19. Код, выполненный в Notebook

```
=== ЕДИНИЧНАЯ МАТРИЦА 4x4 ===
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
Размерность: (4, 4)

=== ДИАГОНАЛЬНАЯ МАТРИЦА 4x4 ===
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]
Размерность: (4, 4)

=== СУММА ЭЛЕМЕНТОВ ЕДИНИЧНОЙ МАТРИЦЫ ===
Сумма всех элементов: 4.0
Сумма элементов на главной диагонали (след): 4.0

=== СУММА ЭЛЕМЕНТОВ ДИАГОНАЛЬНОЙ МАТРИЦЫ ===
Сумма всех элементов: 50
Сумма элементов на главной диагонали (след): 50

=== СРАВНЕНИЕ РЕЗУЛЬТАТОВ ===
Сумма элементов единичной матрицы: 4.0
Сумма элементов диагональной матрицы: 50
Разница: 46.0

=== ДЕТАЛЬНЫЙ АНАЛИЗ ===
Единичная матрица - элементы на диагонали:
[1. 1. 1. 1.]
Единичная матрица - элементы вне диагонали:
Количество нулей вне диагонали: 12

Диагональная матрица - элементы на диагонали:
[ 5 10 15 20]
Диагональная матрица - элементы вне диагонали:
Количество нулей вне диагонали: 12

=== АЛЬТЕРНАТИВНЫЕ СПОСОБЫ СОЗДАНИЯ ===
Единичная через np.identity(4):
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]

Диагональная через np.zeros и fill_diagonal:
[[ 5.  0.  0.  0.]
 [ 0. 10.  0.  0.]
 [ 0.  0. 15.  0.]
 [ 0.  0.  0. 20.]]
```

Рисунок 20. Результат выполнения кода в Notebook

6: Создание и базовые операции с матрицами

Задание:

Создайте две квадратные матрицы NumPy размером 3×3, заполненные случайными целыми числами от 1 до 20. Вычислите и выведите:

- Их сумму
- Их разность
- Их поэлементное произведение

```python
# Упражнение 6 - Создание и базовые операции с матрицами
import numpy as np

# 1. Создаем две матрицы 3x3 со случайными целыми числами от 1 до 20
np.random.seed(42)  # Для воспроизводимости результатов
matrica_A = np.random.randint(1, 21, size=(3, 3))
matrica_B = np.random.randint(1, 21, size=(3, 3))

print("=== МАТРИЦА A ===")
print(matrica_A)
print(f"Размерность: {matrica_A.shape}")
print()

print("=== МАТРИЦА B ===")
print(matrica_B)
print(f"Размерность: {matrica_B.shape}")
print()

# 2. Вычисляем сумму матриц
summa_matric = matrica_A + matrica_B
print("=== СУММА МАТРИЦ (A + B) ===")
print(summa_matric)
print()

# 3. Вычисляем разность матриц
raznost_matric = matrica_A - matrica_B
print("=== РАЗНОСТЬ МАТРИЦ (A - B) ===")
print(raznost_matric)
print()

# 4. Вычисляем поэлементное произведение
poelementnoe_proizvedenie = matrica_A * matrica_B  # Поэлементное умножение
print("=== ПОЭЛЕМЕНТНОЕ ПРОИЗВЕДЕНИЕ (A * B) ===")
print(poelementnoe_proizvedenie)
print()

# 5. Дополнительный анализ
print("=== ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ ===")

# Статистика по матрице A
print("Статистика по матрице A:")
print(f"  Минимальный элемент: {np.min(matrica_A)}")
print(f"  Максимальный элемент: {np.max(matrica_A)}")
print(f"  Среднее значение: {np.mean(matrica_A):.2f}")
print(f"  Сумма всех элементов: {np.sum(matrica_A)}")

# Статистика по матрице B
print("\nСтатистика по матрице B:")
print(f"  Минимальный элемент: {np.min(matrica_B)}")
print(f"  Максимальный элемент: {np.max(matrica_B)}")
print(f"  Среднее значение: {np.mean(matrica_B):.2f}")
print(f"  Сумма всех элементов: {np.sum(matrica_B)}")

# Статистика по сумме
print("\nСтатистика по сумме матриц:")
print(f"  Минимальный элемент: {np.min(summa_matric)}")
print(f"  Максимальный элемент: {np.max(summa_matric)}")
print(f"  Минимальный элемент: {np.min(summa_matric)}")
print(f"  Максимальный элемент: {np.max(summa_matric)}")
print(f"  Среднее значение: {np.mean(summa_matric):.2f}")
print(f"  Сумма всех элементов: {np.sum(summa_matric)}")
print()

# 6. Проверка свойств
print("=== ПРОВЕРКА СВОЙСТВ ===")

# Коммутативность сложения: A + B = B + A
print(f"A + B = B + A? {np.array_equal(matrica_A + matrica_B, matrica_B + matrica_A)}")

# Коммутативность поэлементного умножения: A * B = B * A
print(f"A * B = B * A? {np.array_equal(matrica_A * matrica_B, matrica_B * matrica_A)}")
```

Рисунок 21. Код, выполненный в Notebook

```
=== МАТРИЦА A ===
[[ 7 20 15]
 [11  8  7]
 [19 11 11]]
Размерность: (3, 3)

=== МАТРИЦА B ===
[[ 4  8  3]
 [ 2 12  6]
 [ 2  1 12]]
Размерность: (3, 3)

=== СУММА МАТРИЦ (A + B) ===
[[11 28 18]
 [13 20 13]
 [21 12 23]]

=== РАЗНОСТЬ МАТРИЦ (A - B) ===
[[ 3 12 12]
 [ 9 -4  1]
 [17 10 -1]]

=== ПОЭЛЕМЕНТНОЕ ПРОИЗВЕДЕНИЕ (A * B) ===
[[ 28 160  45]
 [ 22  96  42]
 [ 38  11 132]]

=== ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ ===
Статистика по матрице A:
  Минимальный элемент: 7
  Максимальный элемент: 20
  Среднее значение: 12.11
  Сумма всех элементов: 109

Статистика по матрице B:
  Минимальный элемент: 1
  Максимальный элемент: 12
  Среднее значение: 5.56
  Сумма всех элементов: 50

Статистика по сумме матриц:
  Минимальный элемент: 11
  Максимальный элемент: 28
  Среднее значение: 17.67
```

Рисунок 22. Результат выполнения кода в Notebook

7: Умножение матриц

Задание:

Создайте две матрицы NumPy:

- Первую размером 2×3, заполненную случайными числами от 1 до 10.
- Вторую размером 3×2, заполненную случайными числами от 1 до 10.

Выполните матричное умножение и выведите результат.

```python
# Упражнение 7 - Умножение матриц
import numpy as np

# 1. Создаем матрицы
np.random.seed(42)  # Для воспроизводимости результатов

# Первая матрица: 2 строки, 3 столбца (2×3)
matrica_A = np.random.randint(1, 11, size=(2, 3))
print("=== МАТРИЦА A (2×3) ===")
print(matrica_A)
print(f"Размерность: {matrica_A.shape}")
print(f"Строк: {matrica_A.shape[0]}, Столбцов: {matrica_A.shape[1]}")
print()

# Вторая матрица: 3 строки, 2 столбца (3×2)
matrica_B = np.random.randint(1, 11, size=(3, 2))
print("=== МАТРИЦА B (3×2) ===")
print(matrica_B)
print(f"Размерность: {matrica_B.shape}")
print(f"Строк: {matrica_B.shape[0]}, Столбцов: {matrica_B.shape[1]}")
print()

# Проверка условия умножения
print("=== УСЛОВИЕ УМНОЖЕНИЯ ===")
print(f"Столбцов в A: {matrica_A.shape[1]}")
print(f"Строк в B: {matrica_B.shape[0]}")
print(f"Можно умножать? {matrica_A.shape[1] == matrica_B.shape[0]}")
print()

# 2. Матричное умножение (разные способы)
print("=== МАТРИЧНОЕ УМНОЖЕНИЕ ===")

# Способ 1: оператор @ (рекомендуемый)
rezultat_1 = matrica_A @ matrica_B
print("Способ 1 (A @ B):")
print(rezultat_1)
print(f"Размерность результата: {rezultat_1.shape}")  # Должно быть 2×2
print()
```

```python
rezultat_2 = np.dot(matrica_A, matrica_B)
print("Способ 2 (np.dot(A, B)):")
print(rezultat_2)
print()

# Способ 3: метод .dot()
rezultat_3 = matrica_A.dot(matrica_B)
print("Способ 3 (A.dot(B)):")
print(rezultat_3)
print()

# 3. Детальный расчет для понимания
print("=== ДЕТАЛЬНЫЙ РАСЧЕТ ===")
print("Результирующая матрица C (2×2):")
print("C[0,0] = A[0,0]×B[0,0] + A[0,1]×B[1,0] + A[0,2]×B[2,0]")
print(f"      = {matrica_A[0,0]}×{matrica_B[0,0]} + {matrica_A[0,1]}×{matrica_B[1,0]} + {matrica_A[0,2]}×{matrica_B[2,0]}")
print(f"      = {matrica_A[0,0] * matrica_B[0,0]} + {matrica_A[0,1] * matrica_B[1,0]} + {matrica_A[0,2] * matrica_B[2,0]}")
print(f"      = {rezultat_1[0,0]}")
print()

print("C[0,1] = A[0,0]×B[0,1] + A[0,1]×B[1,1] + A[0,2]×B[2,1]")
print(f"      = {matrica_A[0,0]}×{matrica_B[0,1]} + {matrica_A[0,1]}×{matrica_B[1,1]} + {matrica_A[0,2]}×{matrica_B[2,1]}")
print(f"      = {matrica_A[0,0] * matrica_B[0,1]} + {matrica_A[0,1] * matrica_B[1,1]} + {matrica_A[0,2] * matrica_B[2,1]}")
print(f"      = {rezultat_1[0,1]}")
print()

# 4. Сравнение с поэлементным умножением
print("=== СРАВНЕНИЕ С ПОЭЛЕМЕНТНЫМ УМНОЖЕНИЕМ ===")
print("Поэлементное умножение НЕВОЗМОЖНО, так как размерности не совпадают!")
print("Для поэлементного умножения нужны матрицы одинакового размера.")
print()

# 5. Дополнительные примеры
print("=== ДОПОЛНИТЕЛЬНЫЕ ПРИМЕРЫ ===")

# Транспонирование и умножение
print("A × B =")
print(rezultat_1)
print("\nB^T × A^T =")
print(matrica_B.T @ matrica_A.T)
print("(B^T × A^T) == (A × B)^T?", np.array_equal(matrica_B.T @ matrica_A.T, (rezultat_1).T))
```

Рисунок 23. Код, выполненный в Notebook

```
=== МАТРИЦА A (2×3) ===
[[ 7  4  8]
 [ 5  7 10]]
Размерность: (2, 3)
Строк: 2, Столбцов: 3

=== МАТРИЦА B (3×2) ===
[[3 7]
 [8 5]
 [4 8]]
Размерность: (3, 2)
Строк: 3, Столбцов: 2

=== УСЛОВИЕ УМНОЖЕНИЯ ===
Столбцов в A: 3
Строк в B: 3
Можно умножать? True

=== МАТРИЧНОЕ УМНОЖЕНИЕ ===
Способ 1 (A @ B):
[[ 85 133]
 [111 150]]
Размерность результата: (2, 2)

Способ 2 (np.dot(A, B)):
[[ 85 133]
 [111 150]]

Способ 3 (A.dot(B)):
[[ 85 133]
 [111 150]]

=== ДЕТАЛЬНЫЙ РАСЧЕТ ===
Результирующая матрица C (2×2):
C[0,0] = A[0,0]×B[0,0] + A[0,1]×B[1,0] + A[0,2]×B[2,0]
       = 7×3 + 4×8 + 8×4
       = 21 + 32 + 32
       = 85

C[0,1] = A[0,0]×B[0,1] + A[0,1]×B[1,1] + A[0,2]×B[2,1]
       = 7×7 + 4×5 + 8×8
       = 49 + 20 + 64
```


```
       = 85

C[0,1] = A[0,0]×B[0,1] + A[0,1]×B[1,1] + A[0,2]×B[2,1]
       = 7×7 + 4×5 + 8×8
       = 49 + 20 + 64
       = 133

=== СРАВНЕНИЕ С ПОЭЛЕМЕНТНЫМ УМНОЖЕНИЕМ ===
Поэлементное умножение НЕВОЗМОЖНО, так как размерности не совпадают!
Для поэлементного умножения нужны матрицы одинакового размера.

=== ДОПОЛНИТЕЛЬНЫЕ ПРИМЕРЫ ===
A × B =
[[ 85 133]
 [111 150]]

B^T × A^T =
[[ 85 111]
 [133 150]]
(B^T × A^T) == (A × B)^T? True
```


Рисунок 24. Результат выполнения кода в Notebook

8: Определитель и обратная матрица

Задание:

Создайте случайную квадратную матрицу 3×3. Найдите и выведите:

- Определитель этой матрицы
- Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена)

Используйте функции np.linalg.det и np.linalg.inv.

```python
# Упражнение 8 - Определитель и обратная матрица
import numpy as np

# 1. Создаем случайную квадратную матрицу 3×3
np.random.seed(42)  # Для воспроизводимости результатов
matrica = np.random.randint(1, 11, size=(3, 3))

print("=== ИСХОДНАЯ МАТРИЦА 3×3 ===")
print(matrica)
print(f"Размерность: {matrica.shape}")
print()

# 2. Вычисляем определитель
determinant = np.linalg.det(matrica)
print("=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ ===")
print(f"det(A) = {determinant:.2f}")
print()

# 3. Проверяем, существует ли обратная матрица
# Обратная матрица существует только если определитель ≠ 0
print("=== ПРОВЕРКА СУЩЕСТВОВАНИЯ ОБРАТНОЙ МАТРИЦЫ ===")
if abs(determinant) < 1e-10:  # Проверка с учетом погрешности вычислений
    print("Определитель близок к нулю. Матрица вырождена.")
    print("Обратная матрица НЕ существует.")
else:
    print(f"|det(A)| = {abs(determinant):.2f} ≠ 0")
    print("Обратная матрица СУЩЕСТВУЕТ.")
    print()

    # 4. Вычисляем обратную матрицу
    obratnaya = np.linalg.inv(matrica)
    print("=== ОБРАТНАЯ МАТРИЦА A^(-1) ===")
    print(obratnaya)
    print()

    # 5. Проверка: A × A^(-1) должна равняться единичной матрице
    print("=== ПРОВЕРКА: A × A^(-1) ===")
    proizvedenie = matrica @ obratnaya
    print(proizvedenie)
    print()

    # Округляем для проверки (из-за погрешностей вычислений)
    proizvedenie_okrugl = np.round(proizvedenie, 5)
    print("Проверка (с округлением до 5 знаков):")
    print(proizvedenie_okrugl)

    # Проверяем, близка ли к единичной матрице
    edinichnaya = np.eye(3)
    print(f"\nA × A^(-1) ≈ E ? {np.allclose(proizvedenie, edinichnaya)}")

    # 6. Проверка: A^(-1) × A тоже должна равняться единичной матрице
    print(f"\nA^(-1) × A ≈ E ? {np.allclose(obratnaya @ matrica, edinichnaya)}")
print()

# 7. Дополнительная информация
print("=== ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ ===")
print("Свойства определителя:")
print(f"det(A) = {determinant:.2f}")
print(f"det(A^(-1)) = {np.linalg.det(obratnaya):.2f}")
print(f"det(A) × det(A^(-1)) = {determinant * np.linalg.det(obratnaya):.2f} (должно быть 1)")
```

Рисунок 25. Код, выполненный в Notebook

```
=== ИСХОДНАЯ МАТРИЦА 3×3 ===
[[ 7  4  8]
 [ 5  7 10]
 [ 3  7  8]]
Размерность: (3, 3)

=== ОПРЕДЕЛИТЕЛЬ МАТРИЦЫ ===
det(A) = -26.00

=== ПРОВЕРКА СУЩЕСТВОВАНИЯ ОБРАТНОЙ МАТРИЦЫ ===
|det(A)| = 26.00 ≠ 0
Обратная матрица СУЩЕСТВУЕТ.

=== ОБРАТНАЯ МАТРИЦА A^(-1) ===
[[ 0.53846154 -0.92307692  0.61538462]
 [ 0.38461538 -1.23076923  1.15384615]
 [-0.53846154  1.42307692 -1.11538462]]

=== ПРОВЕРКА: A × A^(-1) ===
[[1.00000000e+00 0.00000000e+00 0.00000000e+00]
 [6.66133815e-16 1.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 1.00000000e+00]]

Проверка (с округлением до 5 знаков):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

A × A^(-1) ≈ E ? True

A^(-1) × A ≈ E ? True

=== ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ ===
Свойства определителя:
det(A) = -26.00
det(A^(-1)) = -0.04
det(A) × det(A^(-1)) = 1.00 (должно быть 1)
```

Рисунок 26. Результат выполнения кода в Notebook

9: Транспонирование и след матрицы

Создайте матрицу NumPy размером 4×4, содержащую случайные целые числа от 1 до 50. Выведите:

- Исходную матрицу
- Транспонированную матрицу
- След матрицы (сумму элементов на главной диагонали)

Используйте np.trace для нахождения следа.

```
# Упражнение 9 - Транспонирование и след матрицы
import numpy as np

# 1. Создаем матрицу 4x4 со случайными целыми числами от 1 до 50
np.random.seed(42)  # Для воспроизводимости результатов
matrica = np.random.randint(1, 51, size=(4, 4))

print("=== ИСХОДНАЯ МАТРИЦА 4x4 ===")
print(matrica)
print(f"Размерность: {matrica.shape}")
print(f"Строк: {matrica.shape[0]}, Столбцов: {matrica.shape[1]}")
print()

# 2. Транспонируем матрицу (разные способы)
print("=== ТРАНСПОНИРОВАННАЯ МАТРИЦА ===")

# Способ 1: атрибут .T
transponirovannaya_1 = matrica.T
print("Способ 1 (matrica.T):")
print(transponirovannaya_1)
print(f"Размерность: {transponirovannaya_1.shape}")
print()

# Способ 2: функция np.transpose()
transponirovannaya_2 = np.transpose(matrica)
print("Способ 2 (np.transpose(matrica)):")
print(transponirovannaya_2)
print()

# Способ 3: метод .transpose()
transponirovannaya_3 = matrica.transpose()
print("Способ 3 (matrica.transpose()):")
print(transponirovannaya_3)
print()

# 3. Вычисляем след матрицы
print("=== СЛЕД МАТРИЦЫ (trace) ===")

# Способ 1: функция np.trace()
sled_1 = np.trace(matrica)
print(f"Способ 1 (np.trace): {sled_1}")
```

Рисунок 27. Код, выполненный в Notebook

```python
# Элементы на главной диагонали
print("Элементы на главной диагонали:")
print(diagonal)
print()

# Визуализация диагонали
print("Матрица с выделенной диагональю:")
for i in range(4):
    row = []
    for j in range(4):
        if i == j:
            row.append(f"[{matrica[i,j]}]")  # Выделяем диагональные элементы
        else:
            row.append(f" {matrica[i,j]} ")
    print(" ".join(row))
print()

# 5. Проверка свойств транспонирования
print("=== ПРОВЕРКА СВОЙСТВ ТРАНСПОНИРОВАНИЯ ===")

# Свойство 1: (A^T)^T = A
print("Свойство 1: (A^T)^T = A")
print(f"Исходная матрица = A")
print(f"(A^T)^T = \n{(matrica.T).T}")
print(f"Равны? {np.array_equal(matrica, (matrica.T).T)}")
print()

# Свойство 2: след A = след A^T
print("Свойство 2: trace(A) = trace(A^T)")
print(f"trace(A) = {np.trace(matrica)}")
print(f"trace(A^T) = {np.trace(matrica.T)}")
print(f"Равны? {np.trace(matrica) == np.trace(matrica.T)}")
print()

# 6. Сравнение исходной и транспонированной
print("=== СРАВНЕНИЕ ИСХОДНОЙ И ТРАНСПОНИРОВАННОЙ ===")
print("Исходная матрица:")
print(matrica)
print("\nТранспонированная матрица:")
print(matrica.T)
print(f"\nСимметричная? {np.array_equal(matrica, matrica.T)}")
```

Рисунок 28. Код, выполненный в Notebook

```
=== ИСХОДНАЯ МАТРИЦА 4x4 ===
[[39 29 15 43]
 [ 8 21 39 19]
 [23 11 11 24]
 [36 40 24  3]]
Размерность: (4, 4)
Строк: 4, Столбцов: 4

=== ТРАНСПОНИРОВАННАЯ МАТРИЦА ===
Способ 1 (matrica.T):
[[39  8 23 36]
 [29 21 11 40]
 [15 39 11 24]
 [43 19 24  3]]
Размерность: (4, 4)

Способ 2 (np.transpose(matrica)):
[[39  8 23 36]
 [29 21 11 40]
 [15 39 11 24]
 [43 19 24  3]]

Способ 3 (matrica.transpose()):
[[39  8 23 36]
 [29 21 11 40]
 [15 39 11 24]
 [43 19 24  3]]

=== СЛЕД МАТРИЦЫ (trace) ===
Способ 1 (np.trace): 74
Способ 2 (matrica.trace()): 74
Способ 3 (ручной расчет): 74

=== ДЕТАЛЬНЫЙ АНАЛИЗ ===
Элементы на главной диагонали:
[39 21 11  3]

Матрица с выделенной диагональю:
[39]  29   15   43
 8  [21]  39   19
 23   11  [11]  24
 36   40   24  [3]
```

Рисунок 29. Результат выполнения кода в Notebook

```
-           -

Матрица с выделенной диагональю:
[39]  29   15   43
 8  [21]  39   19
 23   11  [11]  24
 36   40   24  [3]

=== ПРОВЕРКА СВОЙСТВ ТРАНСПОНИРОВАНИЯ ===
Свойство 1: (A^T)^T = A
Исходная матрица = A
(A^T)^T =
[[39 29 15 43]
 [ 8 21 39 19]
 [23 11 11 24]
 [36 40 24  3]]
Равны? True

Свойство 2: trace(A) = trace(A^T)
trace(A) = 74
trace(A^T) = 74
Равны? True

=== СРАВНЕНИЕ ИСХОДНОЙ И ТРАНСПОНИРОВАННОЙ ===
Исходная матрица:
[[39 29 15 43]
 [ 8 21 39 19]
 [23 11 11 24]
 [36 40 24  3]]

Транспонированная матрица:
[[39  8 23 36]
 [29 21 11 40]
 [15 39 11 24]
 [43 19 24  3]]

Симметричная? False
```

Рисунок 30. Результат выполнения кода в Notebook

10: Системы линейных уравнений

Решите систему линейных уравнений вида:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 5y + 4z = -2 \end{cases}$$

Используйте матричное представление Ax = B, где A — матрица коэффициентов, x — вектор неизвестных, B — вектор правой части. Решите систему с помощью np.linalg.solve и выведите результат.

```python
# Упражнение 10 - Системы линейных уравнений
import numpy as np

# 1. Задаем систему уравнений:
#    2x + 3y - z = 5
#    4x - y + 2z = 6
#    -3x + 5y + 4z = -2

# Матрица коэффициентов A (3×3)
A = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [-3, 5, 4]
])

# Вектор правой части B (3×1)
B = np.array([5, 6, -2])

print("=== СИСТЕМА ЛИНЕЙНЫХ УРАВНЕНИЙ ===")
print("2x + 3y - z = 5")
print("4x - y + 2z = 6")
print("-3x + 5y + 4z = -2")
print()

print("=== МАТРИЧНОЕ ПРЕДСТАВЛЕНИЕ ===")
print("Матрица коэффициентов A:")
print(A)
print(f"\nВектор правой части B:")
print(B)
print()

# 2. Решаем систему с помощью np.linalg.solve
print("=== РЕШЕНИЕ СИСТЕМЫ ===")
x = np.linalg.solve(A, B)
print(f"x = {x[0]:.2f}")
print(f"y = {x[1]:.2f}")
print(f"z = {x[2]:.2f}")
print()

# 3. Проверка решения
print("=== ПРОВЕРКА РЕШЕНИЯ ===")
```

```python
x = np.linalg.solve(A, B)
print(f"x = {x[0]:.2f}")
print(f"y = {x[1]:.2f}")
print(f"z = {x[2]:.2f}")
print()

# 3. Проверка решения
print("=== ПРОВЕРКА РЕШЕНИЯ ===")

# Вычисляем A @ x и сравниваем с B
proverka = A @ x
print("A × x =")
print(proverka)
print("\nДолжно быть равно B =")
print(B)
print()

print("Поэлементная проверка:")
print(f"2x + 3y - z = 2×{x[0]:.2f} + 3×{x[1]:.2f} - {x[2]:.2f} = {proverka[0]:.2f} (должно быть 5)")
print(f"4x - y + 2z = 4×{x[0]:.2f} - {x[1]:.2f} + 2×{x[2]:.2f} = {proverka[1]:.2f} (должно быть 6)")
print(f"-3x + 5y + 4z = -3×{x[0]:.2f} + 5×{x[1]:.2f} + 4×{x[2]:.2f} = {proverka[2]:.2f} (должно быть -2)")
print()

# Проверка с помощью np.allclose
print(f"Решение верно? {np.allclose(A @ x, B)}")
print()

# 4. Альтернативный метод: через обратную матрицу
print("=== АЛЬТЕРНАТИВНЫЙ МЕТОД (через обратную матрицу) ===")

# Находим обратную матрицу
A_obratnaya = np.linalg.inv(A)
print("Обратная матрица A^(-1):")
print(np.round(A_obratnaya, 3))
print()

# Решение: x = A^(-1) × B
x_alt = A_obratnaya @ B
print("Решение через обратную матрицу:")
print(f"x = {x_alt[0]:.2f}")
print(f"y = {x_alt[1]:.2f}")
print(f"z = {x_alt[2]:.2f}")
```

```python
# 5. Дополнительная информация
print("=== ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ ===")

# Определитель матрицы A
det_A = np.linalg.det(A)
print(f"Определитель матрицы A: det(A) = {det_A:.2f}")

# Ранг матрицы A
rank_A = np.linalg.matrix_rank(A)
print(f"Ранг матрицы A: {rank_A} (максимальный ранг = {A.shape[0]})")

# Число обусловленности
cond_A = np.linalg.cond(A)
print(f"Число обусловленности: {cond_A:.2f}")
print("(Показывает, насколько решение чувствительно к погрешностям)")
```

Рисунок 31. Код, выполненный в Notebook

```
=== СИСТЕМА ЛИНЕЙНЫХ УРАВНЕНИЙ ===
2x + 3y - z = 5
4x - y + 2z = 6
-3x + 5y + 4z = -2

=== МАТРИЧНОЕ ПРЕДСТАВЛЕНИЕ ===
Матрица коэффициентов A:
[[ 2  3 -1]
 [ 4 -1  2]
 [-3  5  4]]

Вектор правой части B:
[ 5  6 -2]

=== РЕШЕНИЕ СИСТЕМЫ ===
x = 1.64
y = 0.58
z = 0.01

=== ПРОВЕРКА РЕШЕНИЯ ===
A × x =
[ 5.  6. -2.]

Должно быть равно B =
[ 5  6 -2]

Поэлементная проверка:
2x + 3y - z = 2×1.64 + 3×0.58 - 0.01 = 5.00 (должно быть 5)
4x - y + 2z = 4×1.64 - 0.58 + 2×0.01 = 6.00 (должно быть 6)
-3x + 5y + 4z = -3×1.64 + 5×0.58 + 4×0.01 = -2.00 (должно быть -2)

Решение верно? True

=== АЛЬТЕРНАТИВНЫЙ МЕТОД (через обратную матрицу) ===
Обратная матрица A^(-1):
[[ 0.126  0.153 -0.045]
 [ 0.198 -0.045  0.072]
 [-0.153  0.171  0.126]]

Решение через обратную матрицу:
x = 1.64
y = 0.58
z = 0.01

=== ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ ===
Определитель матрицы A: det(A) = -111.00
Ранг матрицы A: 3 (максимальный ранг = 3)
Число обусловленности: 2.18
(Показывает, насколько решение чувствительно к погрешностям)
```

Рисунок 32. Результат выполнения кода в Notebook

индивидуальному заданию

17. Распределение электроэнергии. Город получает электроэнергию от трех электростанций. Первая станция дает в два раза больше энергии, чем вторая, а третья - на 200 МВт больше, чем вторая. Всего требуется 1500 МВт. Сколько электроэнергии должно поступать с каждой станции?

```python
# Упражнение 17 (Индивидуальное) - Распределение электроэнергии
import numpy as np

print("=== ЗАДАЧА: РАСПРЕДЕЛЕНИЕ ЭЛЕКТРОЭНЕРГИИ ===")
print("Условие:")
print("- Первая станция дает в 2 раза больше энергии, чем вторая")
print("- Третья станция дает на 200 МВт больше, чем вторая")
print("- Общая потребность: 1500 МВт")
print()

# 1. Составляем систему уравнений
# Обозначим:
# x - энергия первой станции
# y - энергия второй станции
# z - энергия третьей станции

# Уравнения:
# 1) x = 2y  →  x - 2y = 0
# 2) z = y + 200  →  -y + z = 200
# 3) x + y + z = 1500

print("=== СИСТЕМА УРАВНЕНИЙ ===")
print("x - 2y = 0      (первая станция дает в 2 раза больше второй)")
print("  - y + z = 200 (третья на 200 МВт больше второй)")
print("x + y + z = 1500 (общая сумма)")
print()

# 2. Матричное представление A × [x,y,z] = B
# A - матрица коэффициентов
# B - вектор правой части

A = np.array([
    [1, -2, 0],    # x - 2y = 0
    [0, -1, 1],    # -y + z = 200
    [1, 1, 1]      # x + y + z = 1500
])

B = np.array([0, 200, 1500])
print("=== МАТРИЧНОЕ ПРЕДСТАВЛЕНИЕ ===")
print("Матрица коэффициентов A:")
print(A)
print("\nВектор правой части B:")
print(B)
print()

# 3. Решаем систему
print("=== РЕШЕНИЕ СИСТЕМЫ ===")
solution = np.linalg.solve(A, B)

x = solution[0]  # первая станция
y = solution[1]  # вторая станция
z = solution[2]  # третья станция

print(f"Первая станция (x): {x:.0f} МВт")
print(f"Вторая станция (y): {y:.0f} МВт")
print(f"Третья станция (z): {z:.0f} МВт")
print()

# 4. Проверка условий
print("=== ПРОВЕРКА УСЛОВИЙ ===")
print(f"1. Первая в 2 раза больше второй? {x:.0f} = 2 × {y:.0f} ? {x == 2*y}")
print(f"   {x:.0f} = {2*y:.0f} ✓")
print()
print(f"2. Третья на 200 больше второй? {z:.0f} = {y:.0f} + 200 ? {z == y + 200}")
print(f"   {z:.0f} = {y + 200:.0f} ✓")
print()
print(f"3. Общая сумма = 1500? {x:.0f} + {y:.0f} + {z:.0f} = {x + y + z:.0f} ? {x + y + z == 1500}")
print(f"   {x + y + z:.0f} = 1500 ✓")
print()

# 5. Детальный вывод
print("=== ОТВЕТ ===")
print(f"Для обеспечения потребности города в 1500 МВт электроэнергии:")
print(f"Первая электростанция: {x:.0f} МВт")
print(f"Вторая электростанция: {y:.0f} МВт")
print(f"Третья электростанция: {z:.0f} МВт")
print()

# 6. Проверка с помощью обратной матрицы (альтернативный метод)
print("=== ПРОВЕРКА ЧЕРЕЗ ОБРАТНУЮ МАТРИЦУ ===")
A_inv = np.linalg.inv(A)
solution_alt = A_inv @ B

print("Решение через A^(-1) × B:")
print(f"Первая: {solution_alt[0]:.0f} МВт")
print(f"Вторая: {solution_alt[1]:.0f} МВт")
print(f"Третья: {solution_alt[2]:.0f} МВт")
print()

# 7. Анализ определителя
det_A = np.linalg.det(A)
print("=== АНАЛИЗ СИСТЕМЫ ===")
print(f"Определитель матрицы A: {det_A:.0f}")
print(f"det(A) ≠ 0? {abs(det_A) > 1e-10} → система имеет единственное решение")
print()

# 8. Визуализация решения (текстовая)
print("=== СХЕМА РАСПРЕДЕЛЕНИЯ ===")
total = x + y + z
print(f"Всего: {total:.0f} МВт")
print()
print(f"Первая:  {'▓' * int(x/50)}{x:.0f} МВт")
print(f"Вторая:  {'▓' * int(y/50)}{y:.0f} МВт")
print(f"Третья:  {'▓' * int(z/50)}{z:.0f} МВт")
```

Рисунок 33. Код, выполненный в Notebook

```
=== ЗАДАЧА: РАСПРЕДЕЛЕНИЕ ЭЛЕКТРОЭНЕРГИИ ===
Условие:
- Первая станция дает в 2 раза больше энергии, чем вторая
- Третья станция дает на 200 МВт больше, чем вторая
- Общая потребность: 1500 МВт

=== СИСТЕМА УРАВНЕНИЙ ===
x - 2y = 0      (первая станция дает в 2 раза больше второй)
  - y + z = 200 (третья на 200 МВт больше второй)
x + y + z = 1500 (общая сумма)

=== МАТРИЧНОЕ ПРЕДСТАВЛЕНИЕ ===
Матрица коэффициентов A:
[[ 1 -2  0]
 [ 0 -1  1]
 [ 1  1  1]]

Вектор правой части B:
[   0  200 1500]

=== РЕШЕНИЕ СИСТЕМЫ ===
Первая станция (x): 650 МВт
Вторая станция (y): 325 МВт
Третья станция (z): 525 МВт

=== ПРОВЕРКА УСЛОВИЙ ===
1. Первая в 2 раза больше второй? 650 = 2 × 325 ? True
   650 = 650 ✓

2. Третья на 200 больше второй? 525 = 325 + 200 ? True
   525 = 525 ✓

3. Общая сумма = 1500? 650 + 325 + 525 = 1500 ? True
   1500 = 1500 ✓

=== ОТВЕТ ===
Для обеспечения потребности города в 1500 МВт электроэнергии:
Первая электростанция: 650 МВт
Вторая электростанция: 325 МВт
Третья электростанция: 525 МВт

=== ПРОВЕРКА ЧЕРЕЗ ОБРАТНУЮ МАТРИЦУ ===
Решение через A^(-1) × B:
Первая: 650 МВт
Вторая: 325 МВт
Третья: 525 МВт

=== АНАЛИЗ СИСТЕМЫ ===
Определитель матрицы A: -4
det(A) ≠ 0? True → система имеет единственное решение

=== СХЕМА РАСПРЕДЕЛЕНИЯ ===
Всего: 1500 МВт

Первая:  ████████████650 МВт
Вторая:  ██████325 МВт
Третья:  ██████████525 МВт
```

Рисунок 34. Результат выполнения кода в Notebook

```
C:\Users\HP\lab-numpy>dir "Lab 2"
 O volume na unidade C é Windows
 O Número de Série do Volume é FEAB-2D4F

 Pasta de C:\Users\HP\lab-numpy\Lab 2

12/03/2026  01:21    <DIR>          .
12/03/2026  01:21    <DIR>          ..
12/03/2026  00:46         2.677.327 Lab 2.docx
               1 arquivo(s)      2.677.327 bytes
               2 pasta(s)    3.824.857.088 bytes disponíveis

C:\Users\HP\lab-numpy>git add "Lab 2/Lab 2.docx"

C:\Users\HP\lab-numpy>git commit -m "Adiciona Laboratório 2 - NumPy"
[main 12a494b] Adiciona Laboratório 2 - NumPy
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Lab 2/Lab 2.docx

C:\Users\HP\lab-numpy>git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 2.27 MiB | 1.15 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Pascoalpm/lab-numpy.git
   71fdceb..12a494b  main -> main
```

Рисунок 35. Файлы загружены в репозиторий Github.

ВЫВОД

В ходе лабораторной работы были изучены основные принципы работы с библиотекой NumPy в Python. Создан и настроен репозиторий GitHub, выполнены практические задания по созданию и обработке массивов, матричным операциям, вычислению определителя и обратной матрицы. Полученные знания применены на практике при решении индивидуального задания (вариант 17) по распределению электроэнергии. Все решения загружены в репозиторий.

1. Каково назначение библиотеки NumPy?

NumPy (Numerical Python) — это библиотека для языка Python, предназначенная для эффективных вычислений с многомерными массивами и матрицами. Она предоставляет высокоуровневые математические функции, оптимизированные по скорости и памяти. NumPy является основой для многих других библиотек: SciPy, Pandas, Matplotlib, Scikit-learn, TensorFlow.

2. Что такое массивы ndarray?

ndarray (n-dimensional array) — это основная структура данных в NumPy. Это многомерный массив, который:

- Хранит элементы одного типа
- Имеет фиксированный размер
- Поддерживает векторизованные операции
- Эффективно использует память (непрерывное хранение)
- Позволяет выполнять операции без явных циклов

3. Как осуществляется доступ к частям многомерного массива?

Доступ к элементам осуществляется через индексацию и срезы (slicing):

```
# Индексация
arr[строка, столбец]
# Срезы
arr[начало:конец:шаг, начало:конец:шаг]
# Примеры:
arr[1, :]      # вся строка с индексом 1
arr[:, 2]      # весь столбец с индексом 2
arr[0:2, 1:3]  # подмассив 2×2
arr[::2]       # каждый второй элемент
```

4. Как осуществляется расчет статистик по данным? NumPy предоставляет множество функций для статистического анализа:

```
np.mean()     # среднее арифметическое
np.median()   # медиана
np.std()      # стандартное отклонение
np.var()      # дисперсия
np.min()      # минимум
np.max()      # максимум
np.sum()      # сумма
np.percentile() # процентили

# С параметром axis можно вычислять по строкам (axis=1) или столбцам (
axis=0)
np.mean(arr, axis=0)  # среднее по столбцам
```

5. Как выполняется выборка данных из массивов ndarray?

Выборка данных осуществляется через:

- Булевы маски: arr[arr > 5] выбирает все элементы >5
- Индексацию: arr[[1, 3, 5]] выбирает элементы по указанным индексам
- Срезы: arr[2:5] выбирает диапазон
- Условия: np.where(arr > 5, arr, 0) заменяет элементы по условию

6. Приведите основные виды матриц и векторов. Опишите способы их создания в языке Python.

Виды векторов:

```
# Вектор-строка
v_row = np.array([1, 2, 3])
# Вектор-столбец
v_col = np.array([[1], [2], [3]])
# Нулевой вектор
v_zeros = np.zeros(5)
# Единичный вектор
v_ones = np.ones(5)
```

Виды матриц:

```
# Квадратная матрица
m_square = np.array([[1, 2], [3, 4]])
# Диагональная матрица
m_diag = np.diag([1, 2, 3, 4])
# Единичная матрица
m_eye = np.eye(4)
# Нулевая матрица
m_zeros = np.zeros((3, 3))
# Матрица с помощью np.matrix (устаревший способ)
m_matrix = np.matrix('1 2 3; 4 5 6')
```

7. Как выполняется транспонирование матриц? Транспонирование — это замена строк матрицы на столбцы. В NumPy есть несколько способов:

```
# Атрибут .T (рекомендуемый)
A_T = A.T
# Функция np.transpose()
A_T = np.transpose(A)
# Метод .transpose()
A_T = A.transpose()
```

8. Приведите свойства операции транспонирования матриц.

1. $(A^T)^T = A$ — двукратное транспонирование возвращает исходную матрицу

2. $(A + B)^T = A^T + B^T$ — транспонирование суммы равно сумме транспонированных

3. $(A \times B)^T = B^T \times A^T$ — транспонирование произведения (порядок меняется)

4. $(\lambda A)^T = \lambda A^T$ — транспонирование произведения на число

5. $\det(A) = \det(A^T)$ — определитель не меняется

6. $\mathrm{trace}(A) = \mathrm{trace}(A^T)$ — след не меняется

9. Какие имеются средства в библиотеке NumPy для выполнения транспонирования матриц?

NumPy предоставляет три основных способа:

- A.T — атрибут (самый простой и быстрый)
- np.transpose(A) — функция
- A.transpose() — метод

10. Какие существуют основные действия над матрицами?

- Сложение и вычитание
- Умножение на число
- Матричное умножение
- Поэлементное умножение
- Транспонирование
- Нахождение определителя
- Нахождение обратной матрицы
- Вычисление ранга
- Нахождение следа

11. Как осуществляется умножение матрицы на число?

Умножение матрицы на число выполняется поэлементно с помощью оператора *:

```
A = np.array([[1, 2], [3, 4]])
lambda_A = 3 * A  # каждый элемент умножается на 3
```

12. Какие свойства операции умножения матрицы на число?

1. $1 \cdot A = A$ — умножение на единицу

2. $0 \cdot A = Z$ — умножение на ноль дает нулевую матрицу

3. $(\alpha + \beta) \cdot A = \alpha \cdot A + \beta \cdot A$ — дистрибутивность относительно суммы чисел

4. $\alpha \cdot (A + B) = \alpha \cdot A + \alpha \cdot B$ — дистрибутивность относительно суммы матриц

5. $(\alpha\beta) \cdot A = \alpha \cdot (\beta \cdot A)$ — ассоциативность

13. Как осуществляется операция сложения и вычитания матриц?

Сложение и вычитание выполняются поэлементно с помощью операторов + и -:

```
C = A + B   # сложение
D = A - B   # вычитание
```

Важно: матрицы должны быть одинаковой размерности.

14. Каковы свойства операции сложения и вычитания матриц?

1. $A + B = B + A$ — коммутативность
2. $A + (B + C) = (A + B) + C$ — ассоциативность
3. $A + Z = A$ — существование нейтрального элемента
4. $A + (-A) = Z$ — существование противоположной матрицы

15. Какие имеются средства в библиотеке NumPy для выполнения операций сложения и вычитания матриц?

NumPy поддерживает стандартные операторы Python:

- A + B — сложение
- A - B — вычитание
- np.add(A, B) — функция сложения
- np.subtract(A, B) — функция вычитания

16. Как осуществляется операция умножения матриц?

Матричное умножение выполняется по правилу: элемент C[i,j] = сумма произведений элементов i-й строки A и j-го столбца B.

Условие: количество столбцов A должно равняться количеству строк B.

```
# Разные способы:
C = A @ B              # оператор @ (рекомендуемый)
C = np.dot(A, B)       # функция dot
C = A.dot(B)           # метод dot
```

17. Каковы свойства операции умножения матриц?

1. A × (B × C) = (A × B) × C — ассоциативность
2. A × (B + C) = A × B + A × C — дистрибутивность слева
3. (B + C) × A = B × A + C × A — дистрибутивность справа
4. A × B ≠ B × A — некоммутативность (в общем случае)
5. E × A = A × E = A — умножение на единичную матрицу
6. Z × A = A × Z = Z — умножение на нулевую матрицу

18. Какие имеются средства в библиотеке NumPy для выполнения операции умножения матриц?

NumPy предоставляет:

- Оператор @ (с Python 3.5)
- Функцию np.dot()
- Метод .dot()
- Функцию np.matmul() (аналогично @)

19. Что такое определитель матрицы? Каковы свойства определителя матрицы?

Определитель (детерминант) — это числовая характеристика квадратной матрицы. Для матрицы 2×2: $\det(A) = a_{11} \cdot a_{22} - a_{12} \cdot a_{21}$

Свойства определителя:

1. $\det(A) = \det(A^T)$ — не меняется при транспонировании
2. $\det(Z) = 0$ — определитель нулевой матрицы равен 0
3. $\det(E) = 1$ — определитель единичной матрицы равен 1

4. При перестановке строк знак меняется на противоположный

5. Если есть нулевая строка/столбец → det = 0

6. Если строки пропорциональны → det = 0

7. $\det(AB) = \det(A) \cdot \det(B)$

8. $\det(A^{-1}) = 1 / \det(A)$

9. Если к строке прибавить другую, умноженную на число → det не меняется

20. Какие имеются средства в библиотеке NumPy для нахождения значения определителя матрицы?

```
import numpy as np
det = np.linalg.det(A)  # вычисление определителя
```

21. Что такое обратная матрица? Какой алгоритм нахождения обратной матрицы?

Обратная матрица A^{-1} — это матрица, удовлетворяющая условию:

$A \times A^{-1} = A^{-1} \times A = E$ (единичная матрица)

Условие существования: $\det(A) \neq 0$ (матрица невырожденная)

Алгоритм нахождения:

1. Найти определитель $|A|$
2. Найти алгебраические дополнения для всех элементов
3. Составить союзную матрицу (из алгебраических дополнений)
4. Транспонировать союзную матрицу (присоединенная матрица)

5. Разделить каждый элемент присоединенной матрицы на определитель

В NumPy это делается одной функцией.

22. Каковы свойства обратной матрицы?

1. $(A^{-1})^{-1} = A$
2. $(A^T)^{-1} = (A^{-1})^T$
3. $(AB)^{-1} = B^{-1} \times A^{-1}$
4. $\det(A^{-1}) = 1 / \det(A)$
5. $E^{-1} = E$ — обратная к единичной равна ей самой

23. Какие имеются средства в библиотеке NumPy для нахождения обратной матрицы?

```
A_inv = np.linalg.inv(A)  # вычисление обратной матрицы
```

24. Самостоятельно изучите метод Крамера для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений методом Крамера средствами библиотеки NumPy.

Метод Крамера использует определители. Для системы $A \cdot x = B$:

1. Вычислить определитель матрицы A: $\Delta = \det(A)$
2. Для каждой переменной x_i создать матрицу A_i, заменив i-й столбец на вектор B
3. Вычислить определитель $\Delta_i = \det(A_i)$
4. $x_i = \Delta_i / \Delta$

Реализация в NumPy:

```
def kramer_method(A, B):
    det_A = np.linalg.det(A)
    if abs(det_A) < 1e-10:
        return None  # система не имеет единственного решения

    n = len(B)
    solutions = []
```

```
    for i in range(n):
        A_i = A.copy()
        A_i[:, i] = B  # заменяем i-й столбец на B
        det_A_i = np.linalg.det(A_i)
        solutions.append(det_A_i / det_A)

    return np.array(solutions)
```

25. Самостоятельно изучите матричный метод для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений матричным методом средствами библиотеки NumPy.

Матричный метод:

Решение системы $A \cdot x = B$ находится как $x = A^{-1} \cdot B$

Алгоритм:

1. Проверить, что $\det(A) \neq 0$
2. Найти обратную матрицу A^{-1}
3. Умножить A^{-1} на вектор B

Реализация в NumPy:

```
def matrix_method(A, B):
    # Способ 1: через обратную матрицу
    det_A = np.linalg.det(A)
    if abs(det_A) < 1e-10:
        return None  # система не имеет единственного решения

    A_inv = np.linalg.inv(A)
    x = A_inv @ B
    return x

# Способ 2: прямая функция solve (рекомендуемый)
x = np.linalg.solve(A, B)  # самый эффективный способ
```

Сравнение методов:

- np.linalg.solve() — самый быстрый и численно устойчивый
- Матричный метод (через обратную матрицу) — требует больше вычислений

- Метод Крамера — нагляден, но неэффективен для больших систем